使用 SlowFast 采样加速扩散大语言模型：三条黄金原则

Qingyan Wei[1] **Yaojie Zhang**[1] **Zhiyuan Liu**[1] **Puyu Zeng**[1]
Yuxuan Wang[1] **Biqing Qi**[2] **Dongrui Liu**[2] **Linfeng Zhang**[1†]

[1]上海交通大学 EPIC 实验室 [2]上海人工智能实验室

代码：https://github.com/LiangrunFlora/Slow-Fast-Sampling

arXiv:2506.10848v3 [cs.CL] 31 Mar 2026

摘要

静态：一直用同一种方法不变

基于扩散的语言模型（dLLM）支持并行生成 token，并显著降低推理延迟，已成为传统自回归大语言模型的有前景替代方案。然而，现有 dLLM 采样策略（如基于置信度或半自回归的解码）常表现出静态行为，导致效率未达最优、灵活性有限。本文提出 SlowFast Sampling，一种在探索阶段和加速解码阶段之间自适应交替的新型动态采样策略。方法由三条黄金原则引导：确定性原则、收敛性原则和位置性原则，它们决定何时、在何处能够有把握且高效地解码 token。我们进一步结合 dLLM-Cache 以减少冗余计算。多种基准上的广泛实验验证了方法的效率。具体而言，在 GPQA 上，SlowFast Sampling 在准确率损失很小的情况下，可使 LLaDA 加速最高 15.63 倍；结合缓存后最高达 34.22 倍。值得注意的是，其吞吐量超过 LLaMA3 8B 等强自回归基线，说明精心设计的采样能够释放 dLLM 快速、高质量生成的全部潜力。

1 引言

大语言模型（LLM）（Zhao et al., 2025）迅速成为人工智能的基石技术，在多种自然语言理解和生成任务中展现出卓越能力。然而，大多数 LLM 普遍采用自回归方式，逐个、顺序生成 token，带来显著推理延迟，尤其在长序列上。为应对这一固有瓶颈，基于扩散的 LLM（dLLM）（Ye et al., 2025; Nie et al., 2025b）作为有前景的替代范式出现。它们能并行生成多个 token，摆脱严格逐 token 生成。并行解码有望显著加快文本生成，使 dLLM 成为高效语言模型推理中引人关注、面向未来的方向。

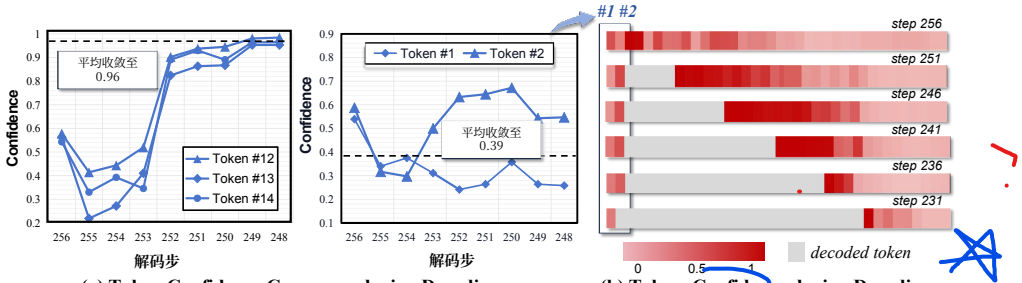
先切固定块，块间从左到右，块内dllm

confidence

但目前 dLLM 的采样方式往往未充分发挥潜力。常见方法包括基于置信度的选择（Chang et al., 2022），例如 Fast-dLLM（Wu et al., 2025）选择超过置信度阈值的 token 解码；另一种流行方法是半自回归解码（Arriola et al., 2025），将序列划为固定块，在块内解码。这些方法常带来不理想的结果，例如大量并行解码时准确率显著下降，且整个生成过程具有静态、恒定的采样速度。缺乏灵活性凸显了动态采样的必要性：应能智能决定每一步采样多少 token，以及它们位于序列的什么位置。

† 表示通信作者。

Chinese translation | original equations, data and references retained



(a) Token Confidence Converges during Decoding (b) Token Confidence during Decoding

越来越确定这个值

图 1：扩散 LLM 采样的三条黄金原则。（a）**收敛性原则：**随着解码进行，多数 token 的置信度收敛至较高值，少数收敛至较低值。（b）256 个扩散步中的置信度图：高置信度 token（深红色）逐渐出现并被优先解码（**确定性原则**），而被选位置往往聚集在连续区域（**位置性原则**），从而支持缓存复用与加速。

置信度高的成片连续出现

置信度高的优先解码

也是qkv，但是是dllm-cache的那一套修改后的理论

针对这些局限，我们提出一种加速 dLLM 的新型动态采样方法，旨在释放其高并行解码的真正潜力。如图 1 所示，方法由三项核心观察引导，我们将其概括为高效加速的三**条黄金原则**：

· **确定性原则：**置信度较高的 token 本身更确定，因此更可能在流程早期被正确解码，在后续扩散步中也更少需要调整。
· **收敛性原则：**随着扩散过程展开和 token 逐步细化，许多 token 的语义趋于稳定，对应置信度分数收敛到稳定值。这意味着其最终形式已基本确定，后续只需极少细化。
· **位置性原则：**即使没有显式约束，模型的采样偏好也往往倾向特定位置的 token，这些位置常彼此相邻或聚集。这种固有位置偏置可被策略性利用，例如有效缓存序列中的部分内容，从而获得显著加速。

保存某些位置此前计算得到的特征

结合上述原则，我们提出包含探索阶段和加速解码阶段的 SlowFast Sampling。探索阶段进行较宽松的解码，定位开始呈现确定性和收敛性的片段；加速阶段再并行解码这些高确定性 token，减少在已确定部分上的计算。该分工带来显著加速：在 GPQA 上，LLaDA 可加速 15.63 倍，与 dLLM-Cache（Liu et al., 2025）结合时最高达 34.22 倍，准确率损失很小。贡献有三点：

1. 提出基于 token 确定性、收敛性及位置影响的三条黄金原则，它们对 dLLM 的有效、高效采样起关键作用。

2. 基于这些原则，提出新型双阶段动态策略 SlowFast Sampling，专门利用这些规律实现 dLLM 的最优加速。

3. 多种基准上的实验表明，SlowFast Sampling 在不损害响应质量的同时显著加速推理，例如 GPQA 上单独使用可使 LLaDA 加速最高 15.63 倍，结合 dLLM-Cache 后最高 34.22 倍；相较基线及更简单的采样方法，取得更优的速度—质量折中。

2 相关工作

2.1 用于语言的扩散模型

扩散模型（DM）（Sohl-Dickstein et al., 2015; Ho et al., 2020; Song et al., 2021）革新了生成建模，尤其在图像等连续领域（Rombach et al.,

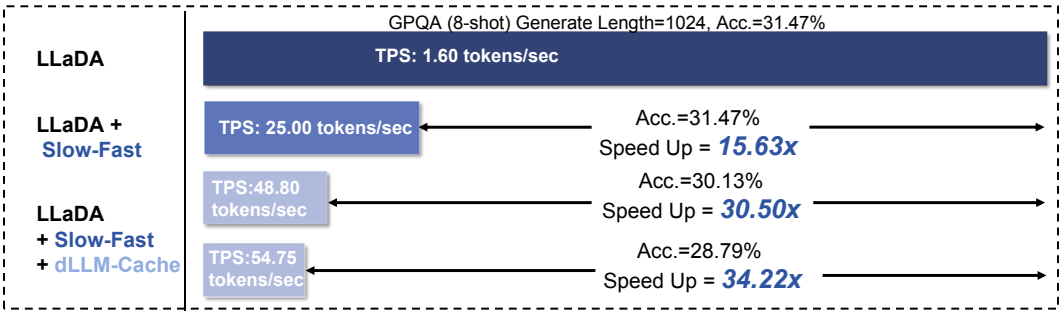


图 2： LLaDA 在 GPQA（8-shot，长度 1024）上的吞吐量和准确率比较，包括：（1）原始解码；（2）SlowFast Sampling；（3）进一步结合 dLLM-Cache 的 SlowFast Sampling。相比原始设置，单独使用 SlowFast Sampling 在保持相近准确率的同时实现 15.63 倍加速。结合缓存后，吞吐量进一步升至 54.75 token/秒（最高 34.22 倍加速），准确率仅有小幅下降。

2022; Peebles & Xie, 2023; Ma et al., 2025b）。但将其用于文本等离散数据时，离散性质带来了独特挑战。离散扩散中一个有前景的方向是掩码扩散模型（MDM）（Austin et al., 2021; Lou et al., 2023; Shi et al., 2024; Nie et al., 2025a;b; Hoogeboom et al., 2021; Campbell et al., 2022），根据上下文迭代预测掩码 token。这些进展改变了文本生成，为 LLM 的自回归范式提供了有吸引力的替代方案。代表包括 LLaDA（Nie et al., 2025b）——使用双向 Transformer 从零训练的 8B MDM——以及由预训练 ARM 权重初始化的 Dream（Ye et al., 2025）。二者性能均可媲美 LLaMA3 8B 等相近规模 ARM（Dubey et al., 2024）。其双向架构可能克服逆向诅咒（Berglund et al., 2023）等 ARM 局限，使扩散成为基础 LLM 的有竞争力方案。

但是本文没有证明，所以原文用的只是“可能”

模型学会了“A 与 B 存在某种关系”，却不一定能反过来从 B 推出 A

“《星河纪事》 → 的作者是 → 李明”

2.2 基于扩散的 LLM 加速方法

dLLM 的高推理延迟主要来自迭代去噪过程（Nie et al., 2025b; Ye et al., 2025），由此推动了加速技术研究。现有策略主要包括缓存机制和先进采样技术。

缓存机制。特征缓存通过复用中间特征减少冗余计算。dLLM-Cache（Liu et al., 2025）将长间隔提示缓存、短间隔响应缓存与 V-verify 机制结合，加快推理。Sparse-dLLM（Song et al., 2025）结合动态缓存驱逐与稀疏注意力，仅保留显著 token，以减少内存并提高速度。dKV-Cache（Ma et al., 2025a）使用延迟 KV 缓存复用已解码 token 的表示，实现 2–10 倍加速。

先进采样技术。优化采样过程本身也是 dLLM 加速的主要方向。低置信度重新掩码（Chang et al., 2022; Nie et al., 2025b）优先处理高置信度 token 以加快收敛；半自回归重新掩码（Nie et al., 2025b; Arriola et al., 2025）将序列分块，应用随机及低置信度策略。此外，GtR（Yan et al., 2026）通过从结构到细节的分层采样加速 MAR；first-hitting sampler 等 MDM 精确模拟方法（Zheng et al., 2024）也在减少采样步数、提高单步效率方面取得进展。

3 方法

3.1 预备知识

扩散大语言模型的推理过程。dLLM 根据提示 $c = (c_1, \cdots, c_M)$，通过迭代去噪生成文本 $y = (y_1, \cdots, y_L)$。模型在 N 个离散步骤中细化中间状态 $y^{(k)} \in T^L$，k 从 N 递减至 0，其中 T 为 token 词表。该过程从全掩码序列开始：

$$\mathbf{y}^{(N)} = (\underbrace{\texttt{[MASK]}, \ldots, \texttt{[MASK]}}_{\text{L 个位置}}) \tag{1}$$

其中 [MASK] ∈ T 是特殊掩码 token。

在每一步 $k \in \{N, N-1, \cdots, 1\}$，掩码预测器 p_θ 根据带噪状态 $y^{(k)}$ 与提示 c，估计原始干净序列 $r_0 = (r_{0,1}, \cdots, r_{0,L})$：

就是取概率最高的

$$P_\theta(\mathbf{r}_0|\mathbf{c}, \mathbf{y}^{(k)}) \tag{2}$$

通过贪心解码得到第 k 步的干净序列估计 $\hat{r}_0^{(k)}$：

$$\hat{r}_{0,i}^{(k)} = \arg\max_{v \in \mathcal{T}} P_\theta(r_{0,i} = v|\mathbf{c}, \mathbf{y}^{(k)}) \quad \forall i \in \{1, \ldots, L\} \tag{3}$$

虽然预测器 p_θ 能在一步中解码全部 [MASK]，但为保证高质量生成，dLLM 采用多步解码。每一步通过重新掩码策略细化 token。向下一状态 $y^{(k-1)}$ 的转移由采样策略 S 控制：

采样策略，决定哪些真正在这一轮接触掩码

$$\mathbf{y}^{(k-1)} = S(\hat{\mathbf{r}}_0^{(k)}, \mathbf{y}^{(k)}, k) \tag{4}$$

这一迭代去噪过程就是采样过程。由于需要 N 个顺序步骤，dLLM 推理计算成本可能很高；本文旨在提高其采样效率。下面以 LLaDA（Nie et al., 2025b）说明核心采样策略，其中时间步定义为 $t_k = k/N$ 和 $t_{k-1} = (k-1)/N$。

LLaDA 为式 4 中的转移函数 S 探索了多种策略，主要区别在于如何用 $\hat{r}_0^{(k)}$ 中的 token 更新 $y^{(k)}$ 并形成 $y^{(k-1)}$，尤其是 $y^{(k)}$ 中原为 [MASK] 的位置：

随机重新掩码。对每个位置 i：若 $y_i^{(k)} \neq$ [MASK]，则 $y_i^{(k-1)} = y_i^{(k)}$，即保留已知 token；若 $y_i^{(k)} =$ [MASK]，则以 $1 - (k-1)/k$ 的概率将 $y_i^{(k-1)}$ 设为 $\hat{r}_{0,i}^{(k)}$，以 $(k-1)/k$ 的概率仍为 [MASK]，使掩码 token 数的期望与噪声调度一致。

置信度低的再重新mask起来

低置信度重新掩码。该确定性策略通过选择性解除掩码提高样本质量。对每个位置 i，若 $y_i^{(k)} =$ [MASK]，模型预测 $\hat{r}_{0,i}^{(k)}$ 并计算其置信度。具体定义为：

$$c_i = P_\theta(\hat{r}_{0,i}^{(k)}|\mathbf{c}, \mathbf{y}^{(k)}). \tag{5}$$

若 $y_i^{(k)} \neq$ [MASK]，则 $c_i = 1$。

状态 $y^{(k-1)}$ 中目标非掩码 token 数为：

$$n_{un} = \lfloor L(1 - t_{k-1}) \rfloor = \lfloor L\left(1 - \frac{k-1}{N}\right) \rfloor. \tag{6}$$

在 $y^{(k-1)}$ 中，对置信度最高的 n_{un} 个 token 解除掩码，其余位置设为 [MASK]。

半自回归重新掩码。LLaDA 在监督微调后使用此策略：将序列分块，从左到右逐块生成；在每块内部，迭代应用随机或低置信度重新掩码，对当前块完成去噪后再处理下一块。

采样策略 S 的选择与步数 N 都会显著影响生成质量和延迟。

3.2 dLLM 的三条黄金原则

基于第 3.1 节的迭代去噪过程，我们对 dLLM，尤其是 LLaDA 等模型的实证分析，揭示了 token 生成中的一致模式。我们将其称为三条黄金原则，作为所提加速策略的基础。它们描述扩散过程中 token 的确定性、收敛性与位置效应如何相互作用，为优化采样提供关键认识。

确定性原则：高置信度意味着高确定性。我们观察到，掩码预测器 p_θ 以高置信度预测的 token，更可能成为

最终正确序列的一部分，且在后续去噪步中倾向于保持不变。位置 i、步骤 k 的预测 token $\hat{r}_{0,i}^{(k)}$ 的置信度为 $P_\theta(\hat{r}_{0,i}^{(k)} \mid c, y^{(k)})$。如图 1(b) 所示，任意步骤 k 中通常都有一部分 token 的置信度显著高于其他 token。这些 token 最适合提前“接受”，或减少重新采样频率。

怎么判断出来的：直接置信概率输出不需要额外的训练

实验证明?
图1

对加速的启示：优先处理快速达到高置信度阈值的 token，可以减少在序列已确定部分上的冗余计算。

收敛性原则：token 随迭代趋于稳定。迭代细化时，单个 token 的预测取值 $\hat{r}_{0,i}^{(k)}$ 及置信度 $P_\theta(\hat{r}_{0,i}^{(k)} \mid c, y^{(k)})$ 都会先波动，再稳定。如图 1(a)，一个典型 token 在早期数个扩散步中可能改变预测值与置信度；随着 k 下降，置信度常收敛至稳定值，说明模型已对当前上下文中的 token 取值形成一致判断。

对加速的启示：已经收敛的 token，即近期窗口内取值和置信度都稳定的 token，后续较少改变。更积极地解码可避免不必要的重新评估，从而加快过程。

提示词，中国首都是哪里，那“北京”及其周围区域会置信度很高

位置性原则：解码具有区域偏好。除单个 token 的行为外，生成过程也常呈现空间模式。高置信度、较早收敛的 token 并非随机散布，而往往出现在连续块或局部区域，见图 1(b)。这可能源于局部语义依赖，或提示 c 中具有强上下文信息部分的影响。例如，经过初始若干步，某个片段的 token 可能整体达到高置信度与稳定性，而其他区域仍大多被掩码或不确定。模型似乎在不同阶段将解码工作集中于不同片段。

没有实验证明

“图书”，相邻位置是“馆”的概率提高

对加速的启示：识别这些区域可实现有针对性的解码；无需均匀处理所有 token，而可将计算资源集中于当前最适合解码的区域。

三条原则共同表明，一刀切的静态采样策略本质上低效。它们促使我们采用动态方法：根据生成序列状态的演化，自适应调整采样 token 数、选择标准和位置。第 3.3 节的 SlowFast Sampling 专门显式利用这些观察，以显著加速推理。

3.3 SlowFast 采样

所以slow阶段也有解码

1. 探索阶段（慢阶段）：识别下一个稳定区域。如图 3 所示，本阶段旨在谨慎推进解码，同时识别适合随后快速处理的稳定区域。从 s_{cycle} 至完整序列末尾 L，在有限的 dLLM 步数内执行：

每进行一次 Slow 探索步，模型就会从尚未解除掩码的位置中，选出最有把握的 \(k_{\text{slow}}\) 个，将它们正式填入序列

· **谨慎解码：**本阶段每一步 k 进行保守更新。在当前探索窗口 $[s_{cycle}, L]$ 内，选择置信度 $P_\theta(\hat{r}_{0,i}^{(k)} \mid c, y^{(k)})$ 最高的 top-k_{slow} 个 token，解除掩码，构成 $\hat{r}_0^{(k)}$ 的一部分（先对这些 token 应用式 3，再应用式 4）。

· **收敛终点预测：**同时预测候选收敛终点 $e_{cand}^{(k)}$，定义为 $[s_{cycle}, L]$ 内预测置信度超过最低阈值 τ_{min_conf} 的最远位置 i。数学形式为：

$$e_{cand}^{(k)} = \max\{i \mid i \in [s_{cycle}, L] \wedge P_\theta(\hat{r}_{0,i}^{(k)} | \mathbf{c}, \mathbf{y}^{(k)}) > \tau_{min_conf}\} \tag{7}$$

$e_{cand}^{(k)}$ 估计当前第 k 步中，模型能有把握地解码到序列中的多远位置。

· **稳定性检查：**探索阶段持续跟踪每一步 j 的候选终点 $e_{cand}^{(j)}$。令 $H_W(k_s) = \{e_{cand}^{(l)} \mid l \in [\max(1, k_s - W_{hist} + 1), k_s]\}$，

方差低于阈值，稳定

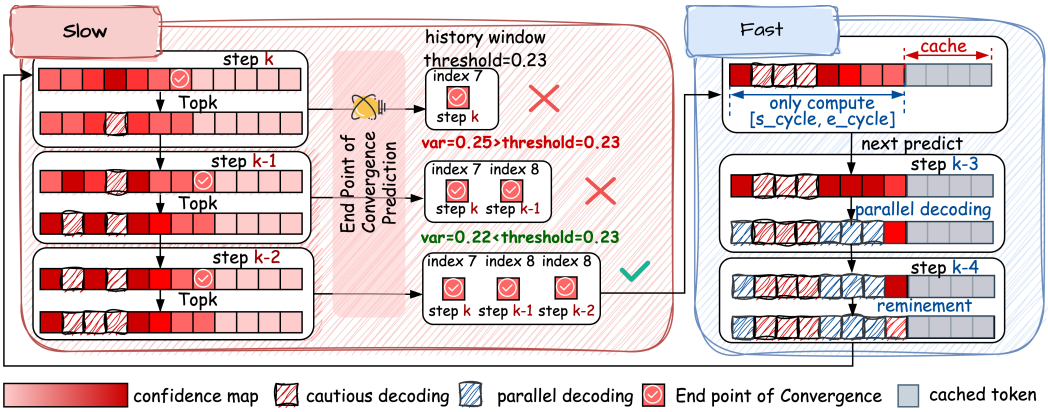


图 3：SlowFast Sampling 流程概览。方法在慢（探索）阶段与快（加速）阶段间交替，以高效生成 token。慢阶段（左）每步选择 top-k 个高置信度 token，谨慎解码，同时持续预测收敛终点并计算历史窗口中的置信度方差。方差低于阈值（如 0.22 < 0.23）时，对应区域 $[s_{cycle}, e_{cycle}]$ 被视为稳定。快阶段（右）对该稳定片段并行解码，积极解除高置信度 token 的掩码；片段外的 token 则暂时跳过，并缓存结果以复用。这种交替结构减少冗余计算，在保持输出质量的同时加速解码。

全部?

就是全部，但是fast可以进行多步直到将整个区域都填完

表示截至探索步 k_s 的最近 W_{hist} 次预测所构成的滑动窗口候选终点集合。当 $k_s \geq W_{hist}$，且 $\mathrm{Var}(H_W(k_s)) < \sigma_{stable}^2$ 时达到稳定，结束本阶段。探索阶段结束步 k_{final} 定义为：

$$k_{final} = \min\left(\{k_s \mid W_{hist} \leq k_s \leq K_{max} \wedge \mathrm{Var}(H_W(k_s)) < \sigma_{stable}^2\} \cup \{K_{max}\}\right) \tag{8}$$

若达到允许的最大探索步数 K_{max} 时仍不满足稳定性标准，则 $k_{final} = K_{max}$。本轮收敛终点 e_{cycle} 设为最终窗口 $H_W(k_{final})$ 内候选终点的均值：

$$e_{cycle} = \mathrm{Mean}(H_W(k_{final})) = \frac{1}{|H_W(k_{final})|} \sum_{e \in H_W(k_{final})} e \tag{9}$$

一旦满足稳定性标准，探索阶段结束。后续加速解码阶段的终点 e_{cycle} 设为最后记录的候选终点 $e_{cand}^{(k)}$。若在最大探索步数内未达到稳定，可将 e_{cycle} 设为保守确定的位置，或在该轮默认对全序列执行谨慎解码。

2. 加速解码阶段（快阶段）：快速并行细化。如图 3 右半部分，一旦识别稳定区域 $[s_{cycle}, e_{cycle}]$，便快速去噪其中的 token，并高效处理区域外 token：

这个缓存只是暂时先不处理，不是直接保存作为结果

· **片段外缓存：**对已识别片段外的 token（$i > e_{cycle}$），若当前预测置信度较低（如低于 τ_{min_conf}），则在本阶段某一步计算其预测值 $\hat{r}_{0,i}^{(k)}$ 并缓存。只要这些位置仍不在活跃解码片段内，后续 dLLM 步便可复用缓存，节省冗余计算。

· **片段内并行解码：**在 $[s_{cycle}, e_{cycle}]$ 内尝试积极的并行解码。对于所有满足 $i \in [s_{cycle}, e_{cycle}]$、$y_i^{(k)} = [\mathrm{MASK}]$ 的 token，只要预测置信度 $P_\theta(\hat{r}_{0,i}^{(k)} \mid c, y^{(k)})$ 超过高确定性阈值 τ_{high_conf}，即：

$$P_\theta(\hat{r}_{0,i}^{(k)} | \mathbf{c}, \mathbf{y}^{(k)}) > \tau_{high_conf} \tag{10}$$

便通过将其值设为对应预测 $\hat{r}_{0,i}^{(k)}$ 来解除掩码。这一更新

在一个概念步骤中对所有符合条件的 token 同时执行，形成 $\hat{r}_0^{(k)}$。

SlowFast 并非对所有任务都有类似的加速效果，最高5.24，最低1.52

加速了分数还提高?

表 1： LLaDA 8B 与 Dream 7B 使用 SlowFast Sampling 后在八个基准上的性能。

任务	方法	推理效率 TPS↑	推理效率 加速比↑	性能 分数↑	方法	推理效率 TPS↑	推理效率 加速比↑	性能 分数↑
数学与科学								
GSM8K	LLaDA	4.55	1.00×	69.83	Dream	8.16	1.00×	77.02
	+ Fast-dLLM (Parallel)	7.45$_{+2.90}$	1.64×$_{+0.64}$	69.60$_{-0.23}$	+ Fast-dLLM (Parallel)	12.72$_{+4.56}$	1.55×$_{+0.55}$	73.09$_{-3.93}$
	+ SlowFast	14.57$_{+10.02}$	3.20×$_{+2.20}$	69.59$_{-0.27}$	**+ SlowFast**	17.15$_{+8.99}$	2.10×$_{+1.10}$	76.50$_{-0.52}$
GPQA	LLaDA	3.31	1.00×	31.47	Dream	5.43	1.00×	35.93
	+ Fast-dLLM (Parallel)	11.72$_{+8.41}$	3.54×$_{+2.54}$	32.13$_{+0.66}$	+ Fast-dLLM (Parallel)	15.88$_{+10.45}$	2.92×$_{+1.92}$	31.01$_{-4.92}$
	+ SlowFast	16.36$_{+13.05}$	4.94×$_{+3.94}$	31.91$_{+0.44}$	**+ SlowFast**	16.56$_{+11.13}$	3.05×$_{+2.05}$	35.94$_{+0.01}$
Math	LLaDA	5.14	1.00×	30.16	Dream	8.48	1.00×	38.68
	+ Fast-dLLM (Parallel)	8.94$_{+3.80}$	1.74×$_{+0.74}$	30.52$_{+0.36}$	+ Fast-dLLM (Parallel)	22.51$_{+14.03}$	2.65×$_{+1.65}$	34.14$_{-4.54}$
	+ SlowFast	11.27$_{+6.13}$	2.19×$_{+1.19}$	29.64$_{-0.52}$	**+ SlowFast**	23.00$_{+14.52}$	2.71×$_{+1.71}$	38.24$_{-0.44}$
通用任务								
MMLU-pro	LLaDA	9.16	1.00×	23.30	Dream	14.97	1.00×	24.14
	+ Fast-dLLM (Parallel)	15.62$_{+6.46}$	1.71×$_{+0.71}$	23.50$_{+0.20}$	+ Fast-dLLM (Parallel)	25.50$_{+10.53}$	1.70×$_{+0.70}$	19.53$_{-4.61}$
	+ SlowFast	23.14$_{+13.98}$	2.53×$_{+1.53}$	23.85$_{+0.55}$	**+ SlowFast**	22.80$_{+7.83}$	1.52×$_{+0.52}$	22.91$_{-1.23}$
MMLU	LLaDA	5.02	1.00×	62.11	Dream	8.46	1.00×	72.61
	+ Fast-dLLM (Parallel)	12.94$_{+7.92}$	2.58×$_{+1.58}$	61.67$_{-0.44}$	+ Fast-dLLM (Parallel)	17.39$_{+8.93}$	2.05×$_{+1.05}$	64.59$_{-8.02}$
	+ SlowFast	16.81$_{+11.79}$	3.35×$_{+2.35}$	66.56$_{+4.45}$	**+ SlowFast**	18.43$_{+9.97}$	2.18×$_{+1.18}$	75.13$_{+2.52}$
BBH	LLaDA	4.04	1.00×	44.97	Dream	6.93	1.00×	51.83
	+ Fast-dLLM (Parallel)	10.73$_{+6.69}$	2.66×$_{+1.66}$	44.13$_{-0.84}$	+ Fast-dLLM (Parallel)	27.84$_{+20.91}$	4.01×$_{+3.01}$	52.28$_{+0.45}$
	+ SlowFast	21.19$_{+17.15}$	5.24×$_{+4.24}$	44.60$_{-0.37}$	**+ SlowFast**	28.14$_{+21.21}$	4.06×$_{+3.06}$	50.55$_{-1.28}$
代码								
MBPP	LLaDA	4.98	1.00×	40.80	Dream	8.92	1.00×	54.20
	+ Fast-dLLM (Parallel)	8.15$_{+3.17}$	1.64×$_{+0.64}$	40.80$_{+0.00}$	+ Fast-dLLM (Parallel)	22.87$_{+13.95}$	2.56×$_{+1.56}$	49.40$_{-4.80}$
	+ SlowFast	13.32$_{+8.34}$	2.67×$_{+1.67}$	41.00$_{+0.20}$	**+ SlowFast**	29.07$_{+20.15}$	3.26×$_{+2.26}$	54.60$_{+0.40}$
HumanEval	LLaDA	11.24	1.00×	31.71	Dream	11.49	1.00×	34.15
	+ Fast-dLLM (Parallel)	23.05$_{+11.81}$	2.05×$_{+1.05}$	32.32$_{+0.61}$	+ Fast-dLLM (Parallel)	24.33$_{+12.84}$	2.12×$_{+1.12}$	32.92$_{-1.23}$
	+ SlowFast	35.46$_{+24.22}$	3.15×$_{+2.15}$	33.54$_{+1.83}$	**+ SlowFast**	25.38$_{+13.89}$	2.21×$_{+1.21}$	35.36$_{+1.21}$

· **回退式 Top-k 细化：** 若片段内达到 $\tau_{\text{high_conf}}$ 标准的 token 数不足以明显推进（如少于一个），则本步在片段内改为保守更新：按置信度选取 $[s_{\text{cycle}}, e_{\text{cycle}}]$ 内 top-k_{fast} 个 token 并解除掩码。这样即便尚未广泛达到高确定性，也能持续推进。

如果没有达到阈值，保底选择topk

因为fast要填完

加速解码阶段完成后，下轮探索阶段起点更新为 $s_{\text{cycle}} \leftarrow e_{\text{cycle}}$。探索与加速解码循环进行，直至生成完整序列。SlowFast 动态调整解码焦点与强度，利用确定性和位置性原则定位合适区域，并利用收敛性原则高效地使其稳定。

4 实验

4.1 实验设置

实现细节。为评估动态采样方法 SlowFast Sampling 的有效性，我们在 LLaDA 8B（Nie et al., 2025b）和 Dream 7B（Ye et al., 2025）等代表性 dLLM 上开展实验，重点测量其在多种基准上的推理加速。所有实验均使用 NVIDIA RTX 4090 GPU。

评估指标。采用定量指标评估 SlowFast Sampling 的采样加速与生成质量。推理速度以每秒 token 数（TPS）衡量，即每秒平均生成多少 token。生成质量使用任务特定指标（例如 GSM8K 准确率），反映加速推理下的模型性能。

4.2 主要结果

跨模型的性能与效率收益。表 1 报告 LLaDA 8B、Dream 7B 使用或不使用 SlowFast Sampling 的吞吐量及模型性能。结果表明，方法显著提高推理效率，并在大多数情况下实现无损加速。关键超参数 $\tau_{\text{min_conf}}$ 与 $\tau_{\text{high_conf}}$ 分别设为 0.1、0.85；其他参数为：最大探索步数 $K_{\max} = 8$，滑动窗口大小

表 2： LLaDA 8B 与 Dream 7B 同时使用 SlowFast Sampling 和 dLLM-Cache 的性能。

任务	方法	推理效率 TPS↑	推理效率 加速比(TPS)↑	性能 分数↑	方法	推理效率 TPS↑	推理效率 加速比(TPS)↑	性能 分数↑
数学与科学								
GSM8K	LLaDA	4.55	1.00×	69.83	Dream	8.16	1.00×	77.02
	+ Fast-dLLM (Parallel+Cache)	15.50$_{+10.95}$	3.41×$_{+2.41}$	68.77$_{-1.06}$	+ Fast-dLLM (Parallel+Cache)	31.07$_{+22.91}$	3.81×$_{+2.81}$	69.45$_{-7.57}$
	+ SlowFast + Cache	26.99$_{+22.44}$	5.93×$_{+4.93}$	69.60$_{-0.23}$	**+ SlowFast + Cache**	46.17$_{+38.01}$	5.66×$_{+4.66}$	72.10$_{-4.92}$
GPQA	LLaDA	3.31	1.00×	31.47	Dream	5.43	1.00×	35.93
	+ Fast-dLLM (Parallel+Cache)	30.21$_{+26.90}$	9.13×$_{+8.13}$	33.03$_{+1.56}$	+ Fast-dLLM (Parallel+Cache)	40.98$_{+35.55}$	7.54×$_{+6.54}$	32.37$_{-3.56}$
	+ SlowFast + Cache	29.06$_{+25.75}$	8.78×$_{+7.78}$	33.48$_{+2.01}$	**+ SlowFast + Cache**	39.63$_{+34.20}$	7.30×$_{+6.30}$	34.82$_{-1.11}$
Math	LLaDA	5.14	1.00×	30.16	Dream	8.48	1.00×	38.68
	+ Fast-dLLM (Parallel+Cache)	21.50$_{+16.36}$	4.18×$_{+3.18}$	28.34$_{-1.82}$	+ Fast-dLLM (Parallel+Cache)	46.80$_{+38.32}$	5.52×$_{+4.52}$	33.24$_{-5.44}$
	+ SlowFast + Cache	26.50$_{+21.36}$	5.16×$_{+4.16}$	29.42$_{-0.74}$	**+ SlowFast + Cache**	56.44$_{+47.96}$	6.66×$_{+5.66}$	37.10$_{-1.58}$
通用任务								
MMLU-pro	LLaDA	9.16	1.00×	23.30	Dream	14.97	1.00×	24.14
	+ Fast-dLLM (Parallel+Cache)	27.85$_{+18.69}$	3.04×$_{+2.04}$	27.66$_{+4.36}$	+ Fast-dLLM (Parallel+Cache)	42.73$_{+27.76}$	2.85×$_{+1.85}$	22.57$_{-1.57}$
	+ SlowFast + Cache	33.38$_{+24.22}$	3.64×$_{+2.64}$	25.53$_{+2.23}$	**+ SlowFast + Cache**	43.50$_{+28.53}$	2.91×$_{+1.91}$	21.81$_{-2.33}$
MMLU	LLaDA	5.02	1.00×	62.11	Dream	8.46	1.00×	72.61
	+ Fast-dLLM (Parallel+Cache)	32.36$_{+27.34}$	6.45×$_{+5.45}$	61.45$_{-0.66}$	+ Fast-dLLM (Parallel+Cache)	44.73$_{+36.27}$	5.28×$_{+4.28}$	64.46$_{-8.15}$
	+ SlowFast + Cache	38.42$_{+33.40}$	7.65×$_{+6.65}$	61.20$_{-0.91}$	**+ SlowFast + Cache**	44.18$_{+35.72}$	5.22×$_{+4.22}$	71.57$_{-1.04}$
BBH	LLaDA	4.04	1.00×	44.97	Dream	6.93	1.00×	51.83
	+ Fast-dLLM (Parallel+Cache)	22.56$_{+18.52}$	5.58×$_{+4.58}$	45.35$_{+0.38}$	+ Fast-dLLM (Parallel+Cache)	57.02$_{+50.09}$	8.22×$_{+7.22}$	44.95$_{-6.88}$
	+ SlowFast + Cache	36.04$_{+32.00}$	8.92×$_{+7.92}$	44.81$_{-0.16}$	**+ SlowFast + Cache**	70.20$_{+63.27}$	10.13×$_{+9.13}$	48.24$_{-3.59}$
代码								
MBPP	LLaDA	4.98	1.00×	40.80	Dream	8.92	1.00×	54.20
	+ Fast-dLLM (Parallel+Cache)	22.18$_{+17.20}$	4.45×$_{+3.45}$	38.20$_{-2.60}$	+ Fast-dLLM (Parallel+Cache)	50.67$_{+41.75}$	5.68×$_{+4.68}$	50.40$_{-3.80}$
	+ SlowFast + Cache	27.26$_{+22.28}$	5.47×$_{+3.87}$	39.00$_{-1.80}$	**+ SlowFast + Cache**	69.48$_{+60.56}$	7.79×$_{+6.79}$	51.00$_{-3.20}$
HumanEval	LLaDA	11.24	1.00×	31.71	Dream	11.49	1.00×	34.15
	+ Fast-dLLM (Parallel+Cache)	26.25$_{+15.01}$	2.34×$_{+1.34}$	29.88$_{-1.83}$	+ Fast-dLLM (Parallel+Cache)	49.47$_{+37.98}$	4.31×$_{+3.31}$	29.27$_{-4.88}$
	+ SlowFast + Cache	41.14$_{+29.90}$	3.66×$_{+2.66}$	31.10$_{-0.61}$	**+ SlowFast + Cache**	47.86$_{+36.37}$	4.17×$_{+3.17}$	35.36$_{+1.21}$

采样策略	TPS↑	分数↑
Autoregressive (AR)	5.25	60.80
Diffusion Sampling	4.55	69.83
Semi-Autoregressive	5.44	66.41
SlowFast Sampling	9.87	69.59

表 3： 不同采样策略在推理效率与生成性能上的比较。

方法	TPS↑	加速比(TPS)↑	Accuracy↑
LLaMA3 8B (Dubey et al., 2024)	33.79	21.12×	31.92
LLaDA	1.60$_{-32.19}$	1.00×	31.47$_{-0.45}$
+ SlowFast	25.00$_{-8.79}$	15.63×	31.47$_{-0.45}$
+ SlowFast + Cache ($K_p = 100, K_r = 5$)	48.80$_{+15.01}$	30.50×	30.13$_{-1.79}$
+ SlowFast + Cache ($K_p = 500, K_r = 30$)	54.75$_{+20.96}$	34.22×	28.79$_{-3.13}$

表 4： LLaDA 8B Base 与其他代表性 LLM 比较。相比 LLaMA3 8B，结合 SlowFast Sampling 与 dLLM-Cache 后的 LLaDA 吞吐量显著更高（最高增加 20.96 TPS），同时保持相近准确率。

$W_{hist} = 2$，稳定方差阈值 $\sigma_{stable}^2 = 1.0$，与第 4.2 节和消融研究中的默认配置一致。

与 dLLM-Cache 的兼容性。近期研究探索了通过特征缓存降低 dLLM 推理计算成本。SlowFast Sampling 与现有缓存机制高度兼容，二者结合比单独使用任一方法能带来更大加速。表 2 比较 LLaDA 使用与不使用本方法和 dLLM-Cache 组合时的性能、推理速度。结果表明，组合可在保持相近模型性能的同时提高吞吐量。

还是静态，但是每次选取所有到达阈值的token，只要达到就全部并行生成

与其他采样策略比较。我们比较 SlowFast Sampling 与四种其他策略：扩散采样、扩散半自回归采样、自回归（AR）采样及 Fast-dLLM（Wu et al., 2025）。扩散采样通过重新掩码策略迭代选择 token 并行解码；AR 严格从左到右；半自回归从左到右逐块生成，在块内应用重新掩码策略。Fast-dLLM 则仅依据 token 置信度进行并行解码，本文同时报告其并行版本和缓存增强版本。表 1、2、3 表明，SlowFast Sampling 在保持有竞争力的生成质量时具有更高推理效率。

4.3 消融研究

SlowFast Sampling 句子生成案例。图 4 展示通过快慢交替生成文本的过程。慢阶段输出少量可靠的 token，例如主语、动词、标点，为句子建立锚点；快阶段在一步中生成较长的高置信度片段，例如“她还剩 9 − 4 = 5 元”，兼顾效率与正确性。这种谨慎慢解码

与有把握的快解码之间的平衡，在加速生成的同时保持准确率，使方法直观而有效。

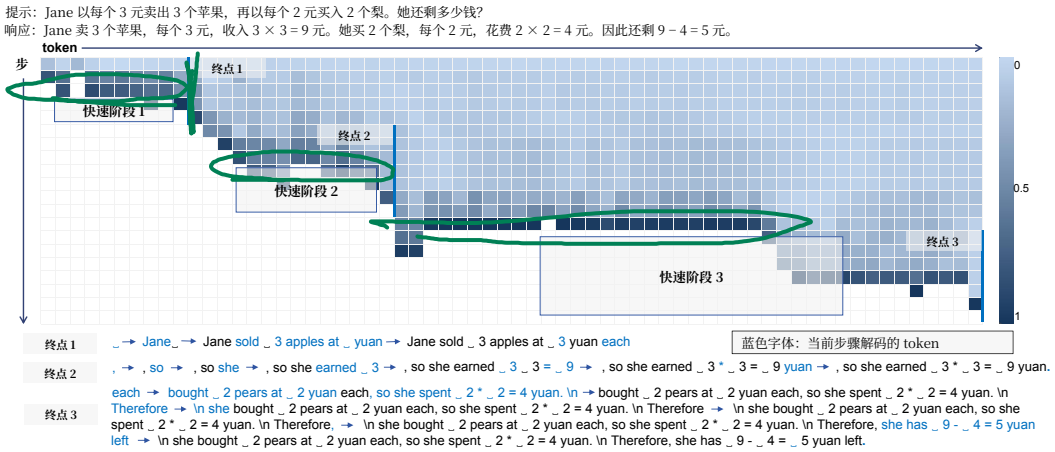


图 4：SlowFast Sampling 句子生成案例。图中展示了三轮探索—加速交替期间置信度图的动态演化。蓝色 token 为当前步骤生成的内容，表明方法如何逐步依托高置信度片段构建连贯的最终响应。

slow阶段每一步预测的终点不一样，
取最后两次

稳定性检查超参数的影响。触发模型由慢阶段切换至快阶段的稳定性检查由三个超参数控制，其影响见图 5。最大探索步数 K_{max} 用于让收敛终点稳定；我们发现 $K_{max} = 8$ 在不过度增加开销的情况下，可提供支持高质量生成的充分探索。滑动窗口大小 W_{hist} 与之配合。预测变化与收敛很快，因此较小窗口 $W_{hist} = 2$ 即有效，在保持质量与最大化推理速度间取得良好平衡。最后，严格的稳定方差阈值 $\sigma_{stable}^2 = 1.0$ 确保只有真正稳定的区域才触发加速，巩固方法可靠性。

slow阶段判断的时候，回看的窗口大小

slow也要多步，去更好的确认边界

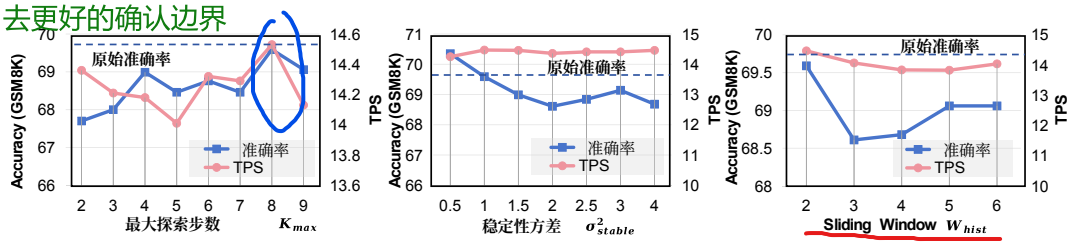


图 5：稳定性检查超参数的敏感性研究。准确率和 TPS 随 K_{max}、σ_{stable}^2、W_{hist} 变化。默认值 $K_{max} = 8$、$\sigma_{stable}^2 = 1.0$、$W_{hist} = 2$ 提供良好的速度—质量折中。

推理速度超越自回归 LLM。表 4 显示，结合 SlowFast Sampling 和 dLLM-Cache 的 LLaDA Base，不仅相较默认配置显著加速，还在保持相近准确率时超越自回归 LLaMA3 8B（Dubey et al., 2024）的推理速度（54.75 对 33.79 TPS）。这说明，经过恰当采样与优化，dLLM 可在效率及实用性方面超越传统自回归模型。

5 结论

本文提出 SlowFast Sampling，一种有原则依据的动态方法，用于加速基于扩散的大语言模型。利用 token 确定性、收敛性和位置偏置三项关键观察，我们设计双阶段解码流程，自适应平衡探索与高效并行解码。广泛基准实验显示，方法不仅显著提高推理速度（GPQA 上最高 15.63 倍，结合 dLLM-Cache 最高 34.22 倍），还保持较强生成质量，吞吐量甚至超过自回归 LLM。我们认为，这是让 dLLM 在真实部署中兼具实用性与竞争力的重要一步。

致谢

本项目由 CCF—腾讯犀牛鸟基金资助。

可复现性声明

我们尽力确保结果可复现。正文第 3–4 节详细介绍 SlowFast Sampling、与 dLLM-Cache 的整合及实验设置。附录 A.3 提供超参数配置和补充实现细节，表 1–4 报告跨基准的全面性能比较。第 4.3 节通过消融研究和案例（如图 4）展示方法稳健性。为方便复现，补充材料包含源代码和实验运行脚本。

伦理声明

本工作聚焦开发 dLLM 的高效采样策略，不涉及人类受试者、个人或敏感数据，以及直接影响个人的应用。所有实验使用公开基准，确保公平性、可复现性和透明度。我们公开代码以促进开放研究并便于验证结果。除语言模型固有的一般问题（如偏见或下游滥用）外，我们尚未发现方法的其他潜在滥用或有害应用；鼓励遵循 ICLR 伦理准则负责任地使用。

参考文献

Marianne Arriola, Aaron Gokaslan, Justin T Chiu, Zhihan Yang, Zhixuan Qi, Jiaqi Han, Subham Sekhar Sahoo, and Volodymyr Kuleshov. Block diffusion: Interpolating between autoregressive and diffusion language models. *arXiv preprint arXiv:2503.09573*, 2025.

Jacob Austin, Daniel D Johnson, Jonathan Ho, Daniel Tarlow, and Rianne Van Den Berg. Structured denoising diffusion models in discrete state-spaces. *Advances in Neural Information Processing Systems*, 34:17981–17993, 2021.

Lukas Berglund, Meg Tong, Max Kaufmann, Mikita Balesni, Asa Cooper Stickland, Tomasz Korbak, and Owain Evans. The reversal curse: Llms trained on" a is b" fail to learn" b is a". *arXiv preprint arXiv:2309.12288*, 2023.

Andrew Campbell, Joe Benton, Valentin De Bortoli, Thomas Rainforth, George Deligiannidis, and Arnaud Doucet. A continuous time framework for discrete denoising models. *Advances in Neural Information Processing Systems*, 35:28266–28279, 2022.

Huiwen Chang, Han Zhang, Lu Jiang, Ce Liu, and William T Freeman. Maskgit: Masked generative image transformer. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 11315–11325, 2022.

Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Amy Yang, Angela Fan, et al. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.

Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. *Advances in neural information processing systems*, 33:6840–6851, 2020.

Emiel Hoogeboom, Didrik Nielsen, Priyank Jaini, Patrick Forré, and Max Welling. Argmax flows and multinomial diffusion: Learning categorical distributions. *Advances in Neural Information Processing Systems*, 34:12454–12465, 2021.

Zhiyuan Liu, Yicun Yang, Yaojie Zhang, Junjie Chen, Chang Zou, Qingyan Wei, Shaobo Wang, and Linfeng Zhang. dllm-cache: Accelerating diffusion large language models with adaptive caching, 2025. URL https://github.com/maomaocun/dLLM-cache.

Aaron Lou, Chenlin Meng, and Stefano Ermon. Discrete diffusion language modeling by estimating the ratios of the data distribution. *arXiv preprint arXiv:2310.16834*, 2023.

Xinyin Ma, Runpeng Yu, Gongfan Fang, and Xinchao Wang. dkv-cache: The cache for diffusion language models, 2025a. URL https://arxiv.org/abs/2505.15781.

Yue Ma, Kunyu Feng, Zhongyuan Hu, Xinyu Wang, Yucheng Wang, Mingzhe Zheng, Xuanhua He, Chenyang Zhu, Hongyu Liu, Yingqing He, et al. Controllable video generation: A survey. *arXiv preprint arXiv:2507.16869*, 2025b.

Shen Nie, Fengqi Zhu, Chao Du, Tianyu Pang, Qian Liu, Guangtao Zeng, Min Lin, and Chongxuan Li. Scaling up masked diffusion models on text, 2025a. URL https://arxiv.org/abs/2410.18514.

Shen Nie, Fengqi Zhu, Zebin You, Xiaolu Zhang, Jingyang Ou, Jun Hu, Jun Zhou, Yankai Lin, Ji-Rong Wen, and Chongxuan Li. Large language diffusion models, 2025b. URL https://arxiv.org/abs/2502.09992.

William Peebles and Saining Xie. Scalable diffusion models with transformers. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pp. 4195–4205, 2023.

Robin Rombach, Andreas Blattmann, Dominik Lorenz, Patrick Esser, and Björn Ommer. High-resolution image synthesis with latent diffusion models. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pp. 10684–10695, 2022.

Jiaxin Shi, Kehang Han, Zhe Wang, Arnaud Doucet, and Michalis K Titsias. Simplified and generalized masked diffusion for discrete data. *arXiv preprint arXiv:2406.04329*, 2024.

Jascha Sohl-Dickstein, Eric Weiss, Niru Maheswaranathan, and Surya Ganguli. Deep unsupervised learning using nonequilibrium thermodynamics. In *International conference on machine learning*, pp. 2256–2265. PMLR, 2015.

Jiaming Song, Chenlin Meng, and Stefano Ermon. Denoising diffusion implicit models. In *International Conference on Learning Representations*, 2021.

Yuerong Song, Xiaoran Liu, Ruixiao Li, Zhigeng Liu, Zengfeng Huang, Qipeng Guo, Ziwei He, and Xipeng Qiu. Sparse-dllm: Accelerating diffusion llms with dynamic cache eviction, 2025. URL https://arxiv.org/abs/2508.02558.

Chengyue Wu, Hao Zhang, Shuchen Xue, Zhijian Liu, Shizhe Diao, Ligeng Zhu, Ping Luo, Song Han, and Enze Xie. Fast-dllm: Training-free acceleration of diffusion llm by enabling kv cache and parallel decoding, 2025. URL https://arxiv.org/abs/2505.22618.

Feihong Yan, Peiru Wang, Yao Zhu, Kaiyu Pang, Qingyan Wei, Huiqi Li, and Linfeng Zhang. Generation then reconstruction: Accelerating masked autoregressive models via two-stage sampling, 2026. URL https://arxiv.org/abs/2510.17171.

Jiacheng Ye, Zhihui Xie, Lin Zheng, Jiahui Gao, Zirui Wu, Xin Jiang, Zhenguo Li, and Lingpeng Kong. Dream 7b, 2025. URL https://hkunlp.github.io/blog/2025/dream.

Wayne Xin Zhao, Kun Zhou, Junyi Li, Tianyi Tang, Xiaolei Wang, Yupeng Hou, Yingqian Min, Beichen Zhang, Junjie Zhang, Zican Dong, Yifan Du, Chen Yang, Yushuo Chen, Zhipeng Chen, Jinhao Jiang, Ruiyang Ren, Yifan Li, Xinyu Tang, Zikang Liu, Peiyu Liu, Jian-Yun Nie, and Ji-Rong Wen. A survey of large language models, 2025. URL https://arxiv.org/abs/2303.18223.

Kaiwen Zheng, Yongxin Chen, Hanzi Mao, Ming-Yu Liu, Jun Zhu, and Qinsheng Zhang. Masked diffusion models are secretly time-agnostic masked models and exploit inaccurate categorical sampling. *arXiv preprint arXiv:2409.02908*, 2024.

A 附录

A.1 大语言模型使用声明

我们仅用大语言模型辅助润色写作、提高语言清晰度，未将其用于研究构思、实验设计、数据分析或科学内容生成。本文的所有研究思想、方法、分析和结论完全由作者完成；作者对论文内容承担全部责任。

min：从当前起点向后寻找，找到预测置信度超过 0.1 的最远 token 位置，作为候选区域终点。

A.2 与先进采样方法的兼容性

TPS每秒生成toke数

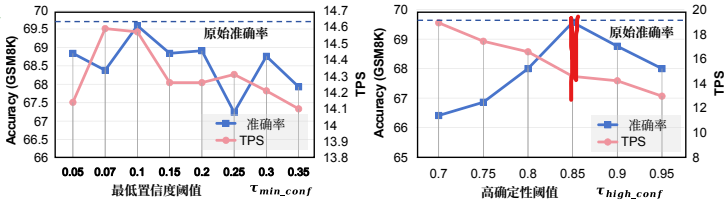


图 6：置信度阈值对 GSM8K 的影响。τ_{min_conf} 控制探索范围，τ_{high_conf} 在快速解码阶段平衡准确率与速度。其他超参数采用第 4.2 节默认设置。

不符合常理：为什么阈值大的ac还降低，论文也没给出

对置信度阈值的敏感性。核心阈值 τ_{min_conf} 和 τ_{high_conf} 决定 SlowFast 流程行为。图 6 中，最低置信度阈值 τ_{min_conf} 定义探索阶段候选区域。适中的 0.1 最优：更低的值带来不稳定区域，更高则形成过于保守、低效的区域。高确定性阈值 τ_{high_conf} 直接控制加速阶段的速度—质量折中；更高的值带来更谨慎、准确的生成，但降低 TPS。我们选择 0.85，在不大幅牺牲速度的情况下取得接近峰值的 GSM8K 分数，实现有效平衡。

A.3 实验细节

为确保公平、可复现的比较，我们标准化各基准的推理生成设置，并按每个模型的官方推荐配置运行。基准特定参数包括推理步数、块长度、总生成长度和少样本示例数。为清楚展示，表 5、表 6 分别汇总 LLaDA 与 Dream 的设置。

表 5： LLaDA 在各基准上的实验设置。

任务	步数	块长度	生成长度	少样本数
MMLU	3	3	3	5
MMLU-pro	256	256	256	0
GSM8K	256	256	256	4
Math	256	256	256	0
GPQA	256	256	256	5
HumanEval	256	256	256	0
MBPP	256	256	256	3
BBH	256	256	256	3

表 6： Dream 在各基准上的实验设置。

任务	步数	生成长度	少样本数
MMLU	3	3	5
MMLU-pro	256	256	0
GSM8K	256	256	8
Math	256	256	4
GPQA	256	256	5
HumanEval	256	256	0
MBPP	256	256	3
BBH	256	256	3

A.4 超参数与稳健性的消融研究

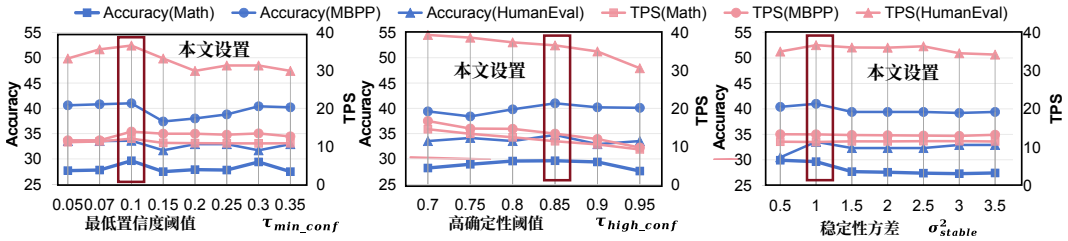


图 7：多基准参数稳健性与敏感性。图中展示 MATH、MBPP、HumanEval 上的敏感性。本文设置在所有任务上都表现出很强的稳健性。

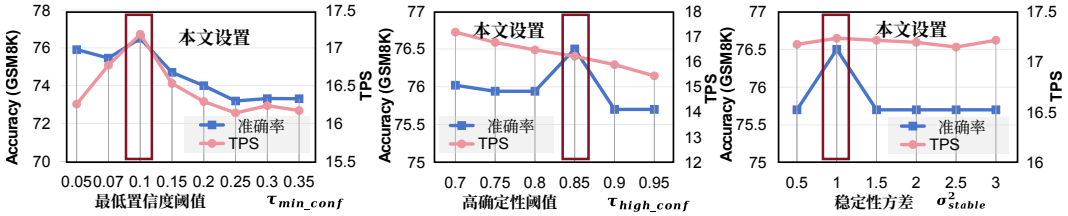


图 8：Dream 7B 模型变体上的参数稳定性。使用 GSM8K，对 Dream 7B 的 τ_{min_conf}、τ_{high_conf} 和 σ_{stable}^2 进行敏感性分析。统一超参数成功迁移到这一要求较高的模型变体，验证了整体参数稳定性。

超参数稳健性与泛化分析。整体分析证实 SlowFast 方法的稳健性和较强泛化能力。图 7 中，一组统一参数可在 MATH、MBPP、HumanEval 等多样基准上保持接近最优的性能；图 8 进一步显示，相同设置可成功迁移到 Dream 7B 的 GSM8K 任务。这说明 SlowFast 具有很强稳健性，不需要为新任务或模型变体进行大量调参。

A.5 不同策略的性能—效率折中

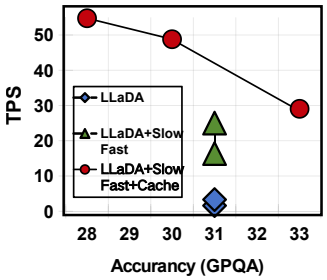


图 9： GPQA 上的性能—效率折中。

图 9 在两种不同评估设置下比较本方法与多种基线的性能—效率。结果证实，整合本方法后，折中曲线相较基线持续明显移向右上方（更高 TPS 与更高准确率）。无论是标准配置（5-shot、256 token），还是要求更高的场景（8-shot、1024 token），都保持这一优势，验证了自适应采样策略的稳健性与可扩展性。

A.6 位置性原则与非连续解码案例研究

对轻微位置不确定性的稳健性。图 10 证实，即使稳定块之间有不稳定区域，SlowFast 仍可覆盖多个稳定块。关键在于，前一稳定块末尾与下一稳定块起点的间距通常较小，自适应机制能高效跳过中间的低置信度片段。因此，即使存在轻微位置不确定性，也能在不牺牲准确率的前提下保持整体加速。

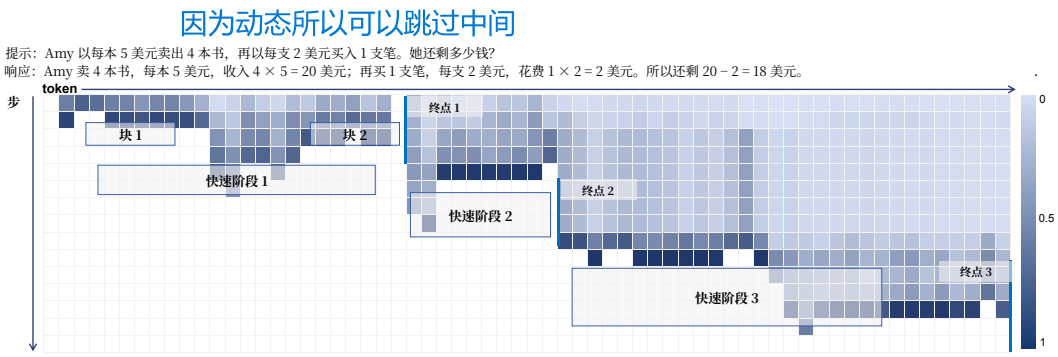


图 10：案例一——位置跳跃时的动态片段覆盖。本例展示已解码 token 之间出现位置跳跃的情况，证实 SlowFast 的动态搜索窗口能够成功识别并覆盖后续稳定区域。

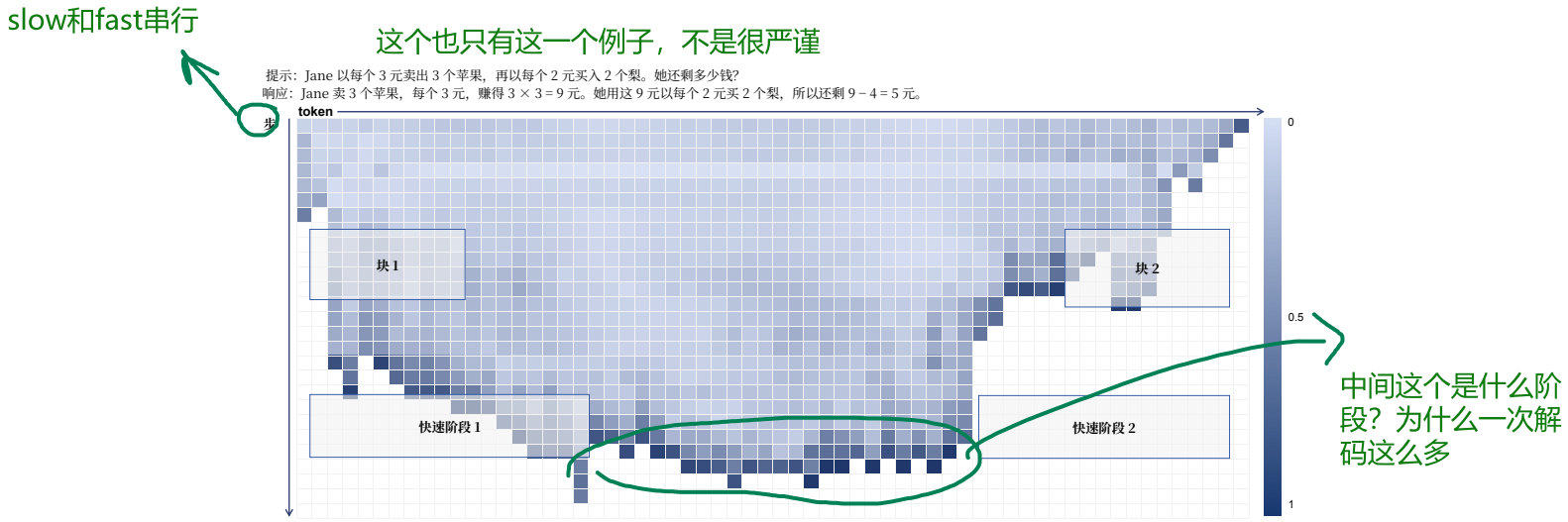


图 11：案例二——SlowFast 对人为诱导的非连续块的自适应响应。通过人为制造位置跳跃，图中展示动态调整机制如何立即识别并转向后续高置信度区域，体现框架的自适应能力。

自适应处理非连续块。图 11 人为构造两个相距较远的高置信度块这一极端场景，严格测试 SlowFast 的自适应边界。可视化表明，遇到两块间的不稳定区域时，动态调整机制能成功识别后续高置信度区域（块 2）并恢复加速（快阶段 2）。这一成功调度展示了方法的稳健性，证实即便在极端条件下，也能自动识别并加速多个非连续稳定块。

A.7 算法：面向 dLLM 的 SlowFast 采样

算法 1 概述 SlowFast Sampling：一个用于加速 dLLM 的动态双阶段框架。首先进入慢速探索阶段，逐步推进（第 10–12 行），确定安全的解码片段边界。随后通过基于方差的稳定性检查（第 16–22 行）验证边界；当终点预测方差低于 σ_{stable}^2 时，确认稳定。接着进入快速加速解码阶段，通过片段内并行解码（第 40–47 行）同时解除高置信度 token 的掩码，实现加速。此外，片段外缓存（第 34–39 行）为下一轮准备。该自适应双阶段机制确保只在可靠稳定的区域实现最大效率。

算法 1　SlowFast 采样策略

输入： 掩码序列 $y^{(N)}$，提示 c，最大步数 N，探索上限 K_{max}，历史窗口 W_{hist}，稳定阈值 ${\sigma_{stable}}^2$，置信度阈值 τ_{min_conf}、τ_{high_conf}。
输出： 生成序列 $y^{(0)}$。

$k \leftarrow N$　▷ 初始化扩散步
$s_{cycle} \leftarrow 1$　▷ 当前解码轮次的起点
$H_W \leftarrow \varnothing$　▷ 收敛终点历史
while $k > 0$ **and** $s_{cycle} < L$ **do**
　// 阶段 1：探索阶段（慢）
　step_count $\leftarrow 0$
　is_stable $\leftarrow$ False
　while step_count $< K_{max}$ **and** $k > 0$ **do**
　　预测 $\hat{r}_0^{(k)}$ 和置信度 $P_\theta(\cdot \mid c, y^{(k)})$
　　谨慎解码：
　　在 $[s_{cycle}, L]$ 内解除 Top-k_{slow} 个 token 的掩码
　　终点预测：
　　$e_{cand}^{(k)} \leftarrow \max\{i \mid i \in [s_{cycle}, L] \wedge P_\theta(\hat{r}_{0,i}^{(k)}) > \tau_{min_conf}\}$
　　用 $e_{cand}^{(k)}$ 更新历史窗口 H_W
　　稳定性检查：
　　$Var_{curr} \leftarrow Variance(H_W)$
　　if $|H_W| \geqslant W_{hist}$ **and** $Var_{curr} < {\sigma_{stable}}^2$ **then**
　　　is_stable $\leftarrow$ True
　　　$e_{cycle} \leftarrow Mean(H_W)$
　　　$k \leftarrow k - 1$
　　　break　▷ 提前退出慢阶段
　　end if
　　使用标准调度器更新状态 $y^{(k-1)}$
　　$k \leftarrow k - 1$
　　step_count $\leftarrow$ step_count + 1
　end while
　if not is_stable **then**
　　$e_{cycle} \leftarrow e_{cand}^{(k)}$　▷ 未收敛时回退
　end if
　// 阶段 2：加速解码阶段（快）
　if $k > 0$ **then**
　　预测 $\hat{r}_0^{(k)}$ 和置信度 $P_\theta(\cdot \mid c, y^{(k)})$
　　片段外缓存：
　　for $i \in (e_{cycle}, L]$ **do**
　　　if $P_\theta(\hat{r}_{0,i}^{(k)}) < \tau_{min_conf}$ **then**
　　　　缓存位置 i 的特征
　　　end if
　　end for
　　片段内并行解码：
　　$S_{high} \leftarrow \{i \mid i \in [s_{cycle}, e_{cycle}] \wedge P_\theta(\hat{r}_{0,i}^{(k)}) > \tau_{high_conf}\}$
　　if $S_{high} \neq \varnothing$ **then**
　　　对全部 $i \in S_{high}$，$y_i^{(k-1)} \leftarrow \hat{r}_{0,i}^{(k)}$　▷ 积极解除掩码
　　else
　　　在 $[s_{cycle}, e_{cycle}]$ 内解除 Top-k_{fast} 个 token 的掩码　▷ 回退细化
　　end if
　　$s_{cycle} \leftarrow e_{cycle}$　▷ 前移稳定区域起点
　　$k \leftarrow k - 1$
　end if
end while
return $y^{(0)}$